DATABASE MANAGEMENT SYSTEM
LAB PROJECT

Gaming Tournament Tracker

A Complete Database Management System

Submitted to:
Ms. Nadia Shakir
Instructor, DBMS Lab

Submitted by:
Abdullah Akbar
03-134242-005
Comp. Sci / 4 - A

Submission Date: 28th April 2026

Examiner Marking Sheet

Deliverable 1 — Project Presentation (5 Marks)

Criteria	Max Marks	Marks Awarded	Comments
Introduction clearly explains purpose, objective, and motivation	1		
Brief and accurate overview of the system and its scope	1		
ERD and Relational Schema explained clearly with visual aids	1		
Normalization steps (1NF, 2NF, 3NF) demonstrated with examples	1		
Front-end / demo shown; system is functional and complete (optional)	1		
SUBTOTAL — Presentation	5		

Deliverable 2 — Source Code (5 Marks)

Criteria	Max Marks	Marks Awarded	Comments
DDL: All tables created with correct SQL data types and constraints	1		
DML: Minimum 10 records per table; INSERT/UPDATE/DELETE implemented	1		
JOINs: INNER, LEFT, RIGHT all implemented and return correct results	1		
Aggregates (COUNT/SUM/AVG/MAX/MIN), Subqueries, and Views all present	1		
Code is properly commented, indented, and follows standard conventions	1		
SUBTOTAL — Source Code	5		

Deliverable 3 — Oral Presentation & Viva (10 Marks)

Criteria	Max Marks	Marks Awarded	Comments
Can explain ERD entities and relationships in own words	2		
Can explain normalization (1NF/2NF/3NF) with project-specific examples	2		
Can explain and modify SQL queries (JOINs, subqueries, aggregates, views)	2		
Can explain SQL data types used and justify each choice	2		
Presentation is clear, confident, and within the 20-minute limit	2		
SUBTOTAL — Viva	10		

Summary	Max	Awarded
Deliverable 1 — Project Presentation	**5**	
Deliverable 2 — Source Code	**5**	
Deliverable 3 — Oral Viva	**10**	
GRAND TOTAL	**20**	

Examiner Signature: _____ Date: _____

Project Proposal

Title of the Project

Gaming Tournament Tracker — A Relational Database Management System

Problem Statement

Competitive gaming tournaments are growing rapidly, yet most small-scale organizers rely on spreadsheets, paper logs, or disconnected tools to manage players, teams, matches, and scores. This leads to data redundancy, inconsistency, and the inability to quickly retrieve meaningful statistics such as leaderboards, player win rates, or team performance history.

There is a need for a structured, relational database system that centralizes all tournament data, eliminates redundancy through normalization, and enables fast querying of results, standings, and statistics through SQL.

Project Scope

This project covers the complete design and implementation of a database system for managing gaming tournaments. The scope includes:

- Database design using Entity Relationship Diagrams (ERD) and Relational Schema
- Normalization up to Third Normal Form (3NF)
- Full SQL implementation using DDL and DML commands
- Sample data population with a minimum of 10 records per table
- Implementation of JOINs, aggregate functions, subqueries, and views
- A web-based front-end interface for interacting with the database
- Documentation covering all phases of the project

Out of scope: Real-time live scoring APIs, mobile applications, and online multiplayer game integration.

List of Functionalities

The Gaming Tournament Tracker will support the following functionalities:

1. Register and manage players with their profiles (name, username, rank, country)
2. Create and manage teams, assigning players to teams
3. Set up tournaments with game type, start/end dates, and prize pool
4. Schedule matches between teams within a tournament
5. Record match scores and determine match winners
6. View real-time leaderboard (implemented as a database VIEW)
7. Calculate player and team statistics: win rate, average score, total matches
8. Search players by username, rank, or country

9. Generate tournament summary reports using aggregate functions
10. Delete or update player, team, or match records

1. Introduction

The Gaming Tournament Tracker is a relational database system designed to manage the complete lifecycle of competitive gaming tournaments. From player registration to match scheduling, score recording, and final standings, the system provides a single, consistent, and normalized data store for all tournament-related information.

This project is built as part of the Database Management System Lab course and demonstrates core DBMS concepts including entity relationship modeling, relational schema design, normalization, and SQL implementation. An optional web-based front-end provides a user-friendly interface to interact with the underlying database.

The system is modeled around a real-world use case — esports tournaments — making it intuitive and engaging for both developers and end users. The data relationships in this domain (players belonging to teams, teams competing in matches, matches belonging to tournaments) provide a rich environment for demonstrating advanced SQL features such as multi-table joins, subqueries, and aggregate-based views.

Motivation

With the rise of esports and gaming communities, even university-level gaming clubs face organizational challenges. A dedicated DBMS solution eliminates manual errors, enables fast data retrieval, and provides insights into performance that simple spreadsheets cannot offer.

2. System Requirements

Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 or equivalent (1.5 GHz+)
RAM	4 GB
Storage	10 GB free disk space
Display	1280 x 720 resolution or higher
Network	Internet connection (for web front-end)

Software Requirements

Software	Version	Purpose
MySQL / MariaDB	8.0+	Database engine
MySQL Workbench	8.0+	ERD design & SQL execution
VS Code	Latest	Code editor
PHP / Node.js	8.0+ / 18+	Back-end for web interface
HTML / CSS / JS	HTML5	Front-end interface
XAMPP / WAMP	Latest	Local server environment

3. ER Diagram

The following section describes the entities, attributes, and relationships in the Gaming Tournament Tracker system. (Draw this diagram in MySQL Workbench or draw.io and paste the image here.)

Entities and Attributes

Entity	Primary Key	Key Attributes
Player	player_id	username, full_name, country, rank_tier, email
Team	team_id	team_name, team_logo, founded_date, captain_id (FK)
Team_Player	player_id + team_id	join_date, role (bridge table)
Game	game_id	game_title, genre, platform, max_team_size
Tournament	tournament_id	tournament_name, game_id (FK), start_date, end_date, prize_pool
Match	match_id	tournament_id (FK), team1_id (FK), team2_id (FK), match_date, venue
Score	score_id	match_id (FK), team_id (FK), points_scored, is_winner

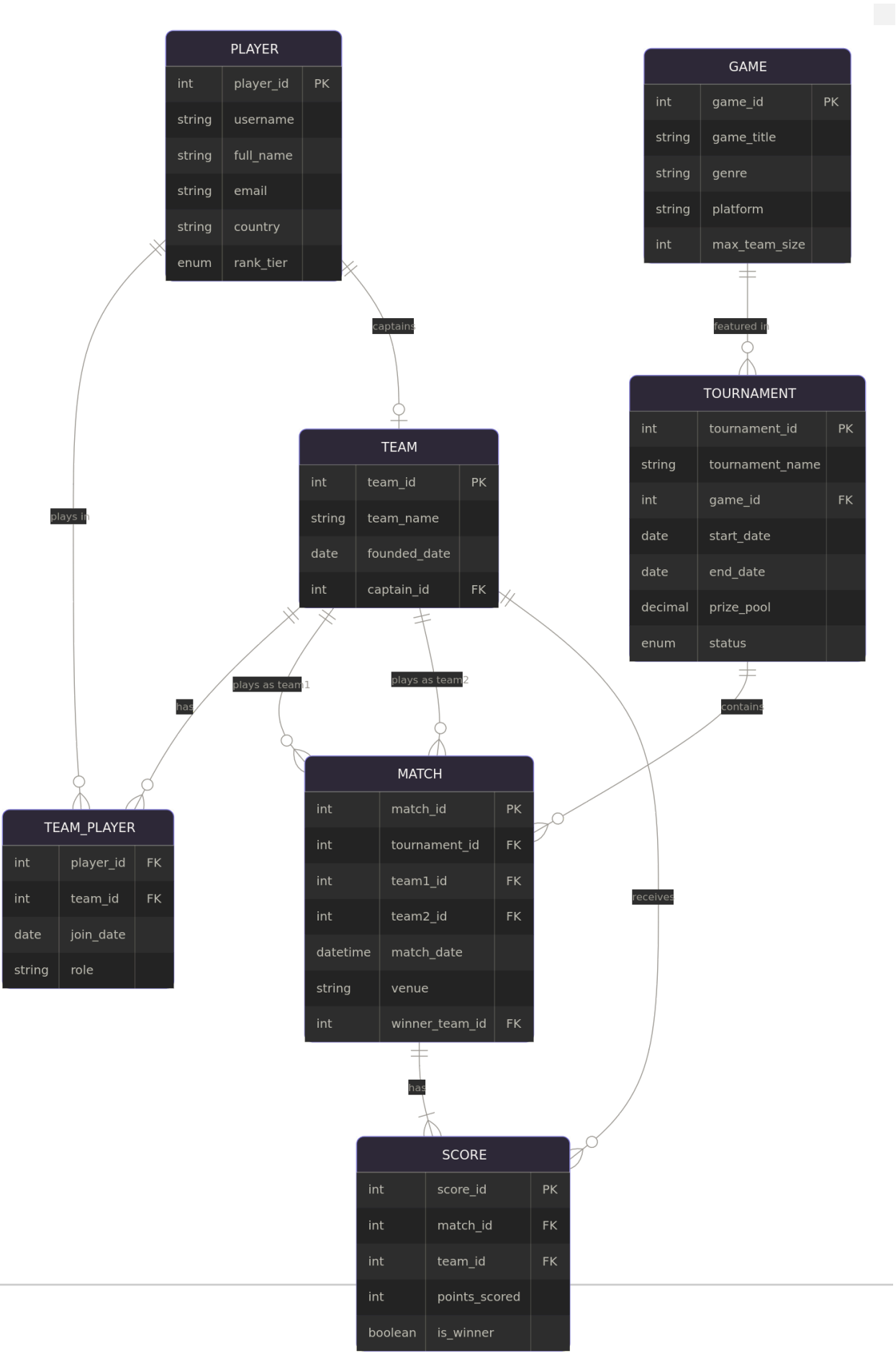
Relationships

- Player — Team: Many-to-Many (a player can be in multiple teams across tournaments; resolved via Team_Player bridge table)
- Team — Match: One Team participates in Many Matches (as team1 or team2)
- Tournament — Match: One Tournament has Many Matches
- Game — Tournament: One Game can have Many Tournaments
- Match — Score: One Match has exactly Two Score records (one per team)

4. Relational Schema

The relational schema derived from the ERD is as follows. Primary keys are underlined and foreign keys are marked with (FK).

```
Player      (player_id PK, username, full_name, email, country, rank_tier)

Team        (team_id PK, team_name, founded_date, captain_id FK→Player.player_id)

Team_Player(player_id FK→Player.player_id, team_id FK→Team.team_id,
```


PLAYER
int player_id PK
string username
string full_name
string email
string country
enum rank_tier
GAME
int game_id PK
string game_title
string genre
string platform
int max_team_size
captains
featured in
TOURNAMENT
int tournament_id PK
string tournament_name
int game_id FK
date start_date
date end_date
decimal prize_pool
enum status
TEAM
int team_id PK
string team_name
date founded_date
int captain_id FK
plays in
plays as team1
plays as team2
has
contains
receives
TEAM_PLAYER
int player_id FK
int team_id FK
date join_date
string role
MATCH
int match_id PK
int tournament_id FK
int team1_id FK
int team2_id FK
datetime match_date
string venue
int winner_team_id FK
has
SCORE
int score_id PK
int match_id FK
int team_id FK
int points_scored
boolean is_winner

```
join_date, role)   ← Composite PK: (player_id, team_id)

Game        (game_id PK, game_title, genre, platform, max_team_size)

Tournament (tournament_id PK, tournament_name, game_id FK→Game.game_id,
            start_date, end_date, prize_pool, status)

Match       (match_id PK, tournament_id FK→Tournament.tournament_id,
            team1_id FK→Team.team_id, team2_id FK→Team.team_id,
            match_date, venue, winner_team_id FK→Team.team_id)

Score       (score_id PK, match_id FK→Match.match_id,
            team_id FK→Team.team_id, points_scored, is_winner)
```

5. Normalization Steps

All tables in the Gaming Tournament Tracker are normalized up to Third Normal Form (3NF). The following steps demonstrate the normalization process.

First Normal Form (1NF)

Rule: Each column must contain atomic values, no repeating groups.

Problem (Unnormalized): A single Match record stored team names and scores as comma-separated values in one column:

```
Match_Unnormalized:
match_id | teams              | scores    | tournament
1        | TeamA, TeamB       | 15, 8     | PUBG Cup 2025
2        | TeamC, TeamD       | 20, 20    | PUBG Cup 2025
```

Fix: Split into atomic columns — separate team1_id, team2_id, and create a Score table for individual scores.

Second Normal Form (2NF)

Rule: Must be in 1NF and every non-key attribute must be fully dependent on the entire primary key (no partial dependencies).

Problem: In an early version of Team_Player, tournament_name was included but it depends only on tournament_id, not on the composite key (player_id, team_id):

```
Team_Player_Bad:
(player_id, team_id) → join_date, role, tournament_name  ← WRONG
tournament_name depends only on tournament, not on this composite key
```

Fix: Remove tournament_name from Team_Player. Tournament information belongs in the Tournament table.

Third Normal Form (3NF)

Rule: Must be in 2NF and there must be no transitive dependencies (non-key attributes depending on other non-key attributes).

Problem: If we stored game_genre inside the Tournament table, it would create a transitive dependency: tournament_id → game_id → game_genre.

```
Tournament_Bad:
tournament_id → tournament_name, game_id, game_genre  ← WRONG
game_genre transitively depends on game_id, not directly on tournament_id
```

Fix: Remove game_genre from Tournament. It belongs in the Game table. The Tournament table references game_id as a foreign key and all genre information is retrieved via JOIN.

Result: All 7 tables are now in 3NF — no redundancy, no partial dependencies, no transitive dependencies.

6. SQL Queries

6.1 DDL — Create Tables

```
CREATE DATABASE gaming_tracker;
USE gaming_tracker;

CREATE TABLE Player (
  player_id    INT AUTO_INCREMENT PRIMARY KEY,
  username     VARCHAR(50) UNIQUE NOT NULL,
  full_name    VARCHAR(100) NOT NULL,
  email        VARCHAR(100) UNIQUE NOT NULL,
  country      VARCHAR(50),
  rank_tier    ENUM('Bronze','Silver','Gold','Platinum','Diamond') DEFAULT 'Bronze'
);

CREATE TABLE Team (
  team_id      INT AUTO_INCREMENT PRIMARY KEY,
  team_name    VARCHAR(100) UNIQUE NOT NULL,
  founded_date DATE,
  captain_id   INT,
  FOREIGN KEY (captain_id) REFERENCES Player(player_id)
);

CREATE TABLE Team_Player (
  player_id    INT NOT NULL,
  team_id      INT NOT NULL,
  join_date    DATE,
  role         VARCHAR(50),
  PRIMARY KEY (player_id, team_id),
  FOREIGN KEY (player_id) REFERENCES Player(player_id),
  FOREIGN KEY (team_id)   REFERENCES Team(team_id)
);

CREATE TABLE Game (
  game_id      INT AUTO_INCREMENT PRIMARY KEY,
  game_title   VARCHAR(100) NOT NULL,
  genre        VARCHAR(50),
  platform     VARCHAR(50),
  max_team_size INT
);

CREATE TABLE Tournament (
  tournament_id   INT AUTO_INCREMENT PRIMARY KEY,
  tournament_name VARCHAR(150) NOT NULL,
  game_id         INT,
  start_date      DATE,
  end_date        DATE,
  prize_pool      DECIMAL(10,2),
  status          ENUM('Upcoming','Ongoing','Completed') DEFAULT 'Upcoming',
  FOREIGN KEY (game_id) REFERENCES Game(game_id)
);

CREATE TABLE Match (
  match_id        INT AUTO_INCREMENT PRIMARY KEY,
  tournament_id   INT,
  team1_id        INT,
  team2_id        INT,
  match_date      DATETIME,
  venue           VARCHAR(100),
```

```
  winner_team_id  INT,
  FOREIGN KEY (tournament_id)  REFERENCES Tournament(tournament_id),
  FOREIGN KEY (team1_id)       REFERENCES Team(team_id),
  FOREIGN KEY (team2_id)       REFERENCES Team(team_id),
  FOREIGN KEY (winner_team_id) REFERENCES Team(team_id)
);

CREATE TABLE Score (
  score_id      INT AUTO_INCREMENT PRIMARY KEY,
  match_id      INT,
  team_id       INT,
  points_scored INT DEFAULT 0,
  is_winner     BOOLEAN DEFAULT FALSE,
  FOREIGN KEY (match_id) REFERENCES Match(match_id),
  FOREIGN KEY (team_id)  REFERENCES Team(team_id)
);
```

6.2 DML — Insert Sample Data

```
-- Players (10 records)
INSERT INTO Player (username, full_name, email, country, rank_tier) VALUES
  ('xShadow',   'Ali Hassan',     'ali@mail.com',    'Pakistan', 'Diamond'),
  ('BladeX',    'Bilal Khan',     'bilal@mail.com',  'Pakistan', 'Platinum'),
  ('FireStorm', 'Sara Ahmed',     'sara@mail.com',   'Pakistan', 'Gold'),
  ('NightOwl',  'Umar Farooq',    'umar@mail.com',   'India',    'Gold'),
  ('ZeroGrav',  'Rania Siddiq',   'rania@mail.com',  'UAE',      'Silver'),
  ('GhostByte', 'Hamza Malik',    'hamza@mail.com',  'Pakistan', 'Platinum'),
  ('TurboX',    'Kamran Shah',    'kamran@mail.com', 'Pakistan', 'Bronze'),
  ('LazorEye',  'Farrukh Aziz',   'faz@mail.com',   'Uzbekistan','Diamond'),
  ('StormRider','Noor Bibi',      'noor@mail.com',   'Pakistan', 'Silver'),
  ('VoidWalker','Tariq Mehmood',  'tariq@mail.com',  'Pakistan', 'Gold');

-- Teams (10 records)
INSERT INTO Team (team_name, founded_date, captain_id) VALUES
  ('Alpha Squad',    '2023-01-15', 1),
  ('Blade Runners',  '2023-03-10', 2),
  ('Storm Chasers',  '2023-06-20', 3),
  ('Night Wolves',   '2022-11-05', 4),
  ('Zero Protocol',  '2024-01-01', 5),
  ('Ghost Unit',     '2023-08-14', 6),
  ('Turbo Force',    '2024-02-28', 7),
  ('Lazor Squad',    '2022-09-09', 8),
  ('Storm Brigade',  '2023-12-12', 9),
  ('Void Breakers',  '2024-03-03', 10);
```

6.3 DML — UPDATE and DELETE

```
-- Update a player's rank
UPDATE Player
SET rank_tier = 'Diamond'
WHERE username = 'BladeX';

-- Update tournament status
UPDATE Tournament
SET status = 'Completed'
WHERE tournament_id = 1;

-- Delete a match record
```

```
DELETE FROM Score WHERE match_id = 5;
DELETE FROM Match WHERE match_id = 5;
```

6.4 SELECT Queries with JOINs

```
-- INNER JOIN: Show all matches with team names
SELECT m.match_id, t1.team_name AS team1, t2.team_name AS team2,
       m.match_date, tn.tournament_name
FROM Match m
INNER JOIN Team t1 ON m.team1_id = t1.team_id
INNER JOIN Team t2 ON m.team2_id = t2.team_id
INNER JOIN Tournament tn ON m.tournament_id = tn.tournament_id;

-- LEFT JOIN: Show all teams and their match count (including teams with 0 matches)
SELECT t.team_name, COUNT(m.match_id) AS total_matches
FROM Team t
LEFT JOIN Match m ON t.team_id = m.team1_id OR t.team_id = m.team2_id
GROUP BY t.team_name;

-- RIGHT JOIN: All tournaments, even those without matches scheduled yet
SELECT tn.tournament_name, m.match_date
FROM Match m
RIGHT JOIN Tournament tn ON m.tournament_id = tn.tournament_id;
```

6.5 Aggregate Functions

```
-- Total prize pool across all tournaments
SELECT SUM(prize_pool) AS total_prize_pool FROM Tournament;

-- Average points scored per team in all matches
SELECT t.team_name, AVG(s.points_scored) AS avg_score
FROM Score s
INNER JOIN Team t ON s.team_id = t.team_id
GROUP BY t.team_name
ORDER BY avg_score DESC;

-- Count of players per country
SELECT country, COUNT(*) AS player_count
FROM Player
GROUP BY country
ORDER BY player_count DESC;

-- Highest scoring team in a tournament
SELECT t.team_name, SUM(s.points_scored) AS total_points
FROM Score s
JOIN Team t ON s.team_id = t.team_id
JOIN Match m ON s.match_id = m.match_id
WHERE m.tournament_id = 1
GROUP BY t.team_name
ORDER BY total_points DESC
LIMIT 1;
```

6.6 Subqueries

```
-- Teams that have never lost a match (won all matches they played)
SELECT team_name FROM Team
```

```
WHERE team_id NOT IN (
  SELECT team_id FROM Score WHERE is_winner = FALSE
);

-- Players on teams that participated in the most expensive tournament
SELECT p.username, p.full_name, t.team_name
FROM Player p
JOIN Team_Player tp ON p.player_id = tp.player_id
JOIN Team t ON tp.team_id = t.team_id
WHERE t.team_id IN (
  SELECT DISTINCT team1_id FROM Match WHERE tournament_id = (
    SELECT tournament_id FROM Tournament
    ORDER BY prize_pool DESC LIMIT 1
  )
);
```

6.7 Views

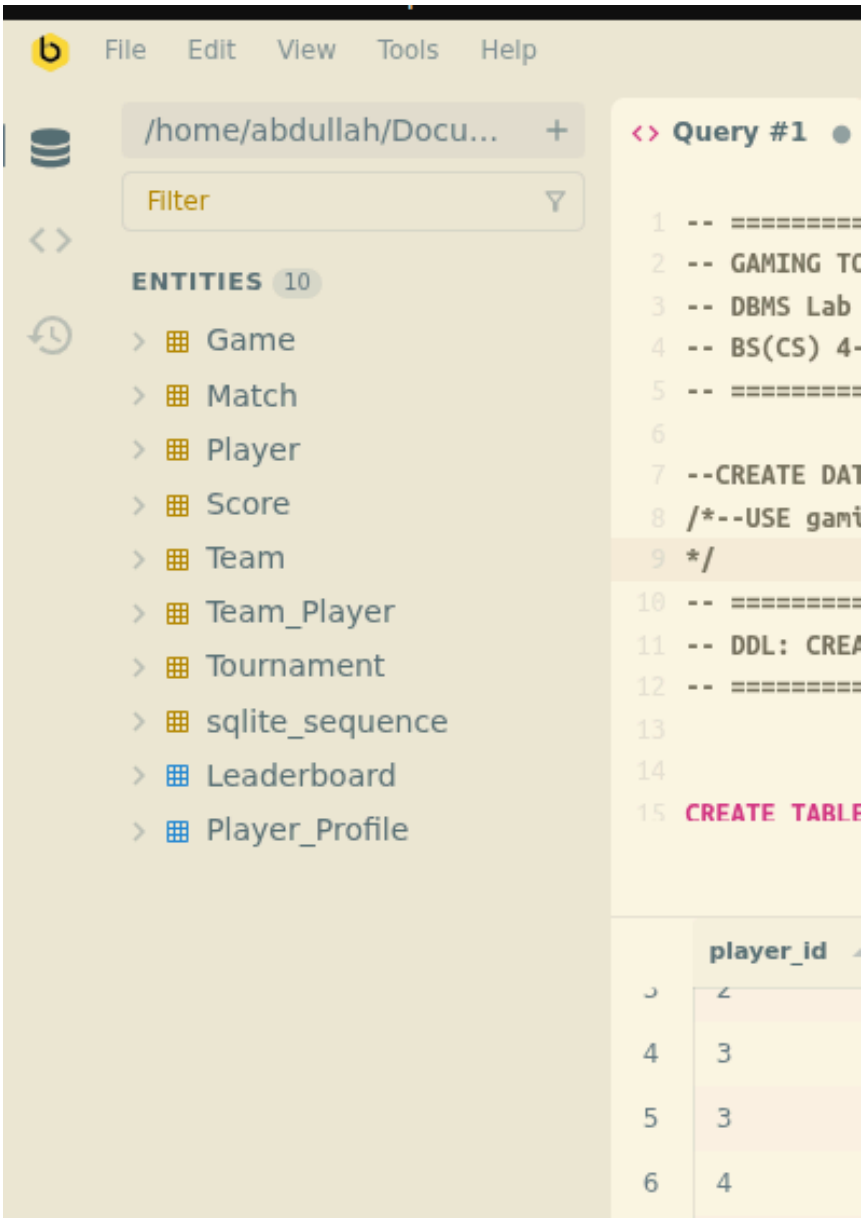
```
-- View 1: Tournament Leaderboard (live-recalculated every time it is queried)
CREATE VIEW Leaderboard AS
SELECT t.team_name,
       COUNT(CASE WHEN s.is_winner = TRUE THEN 1 END) AS wins,
       COUNT(CASE WHEN s.is_winner = FALSE THEN 1 END) AS losses,
       SUM(s.points_scored) AS total_points
FROM Team t
LEFT JOIN Score s ON t.team_id = s.team_id
GROUP BY t.team_name
ORDER BY wins DESC, total_points DESC;

-- Query the view
SELECT * FROM Leaderboard;

-- View 2: Player Full Profile with team info
CREATE VIEW Player_Profile AS
SELECT p.player_id, p.username, p.full_name, p.country, p.rank_tier,
       t.team_name, tp.role
FROM Player p
LEFT JOIN Team_Player tp ON p.player_id = tp.player_id
LEFT JOIN Team t ON tp.team_id = t.team_id;

SELECT * FROM Player_Profile WHERE country = 'Pakistan';
```

7. Screenshots of the Working System

The following screenshots demonstrate the working system. Replace each placeholder with actual screenshots taken from your MySQL Workbench or web interface.

*[Screenshot 1: MySQL Workbench — ERD Diagram view] : **pasted above***

[Screenshot 2: CREATE TABLE]



[Screenshot 3: SELECT query result showing the Leaderboard VIEW]

```
276 -- Query the views
277 SELECT * FROM Leaderboard;
```

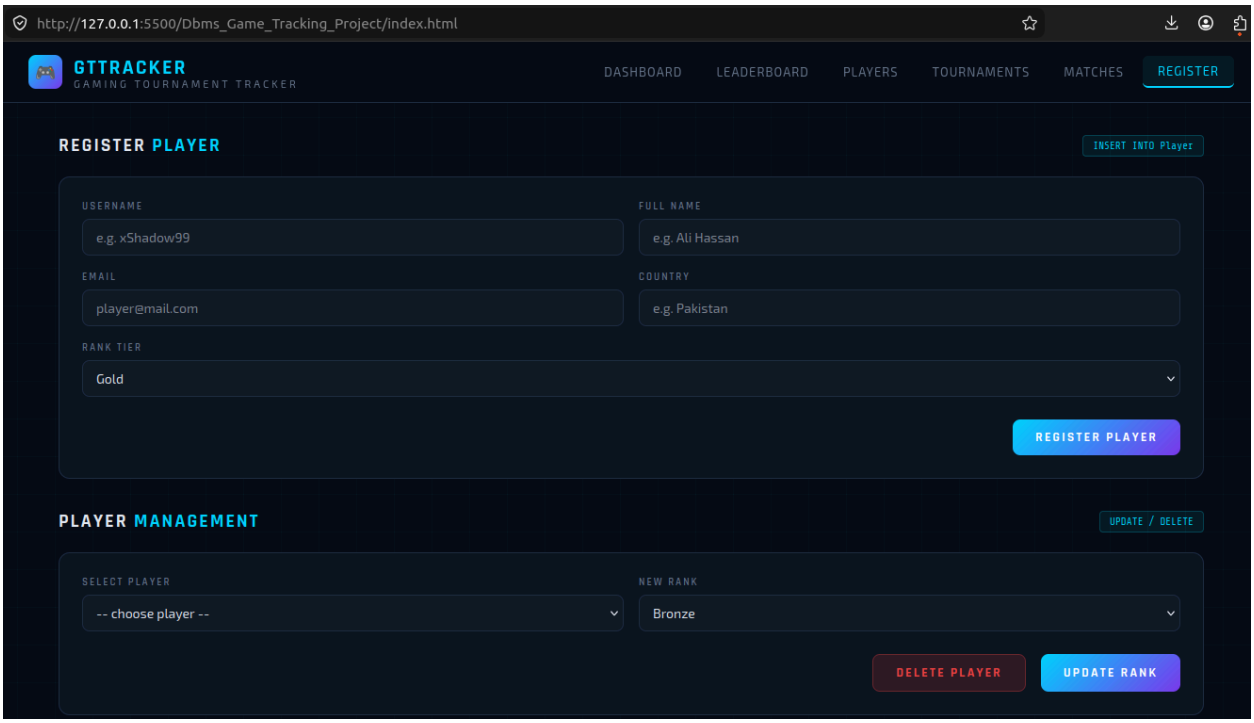
	team_name	wins	losses	total_points	avg_score
1	Alpha Squad	3	0	66	22
2	Lazor Squad	2	0	37	18.5
3	Zero Protocol	2	0	34	17
4	Storm Chasers	1	1	36	18
5	Storm Brigade	1	1	35	17.5
6	Ghost Unit	1	1	29	14.5
7	Turbo Force	0	2	23	11.5
8	Blade Runners	0	2	20	10
9	Void Breakers	0	2	20	10
10	Night Wolves	0	1	9	9

[Screenshot 4: INNER JOIN query result showing match details with team names]

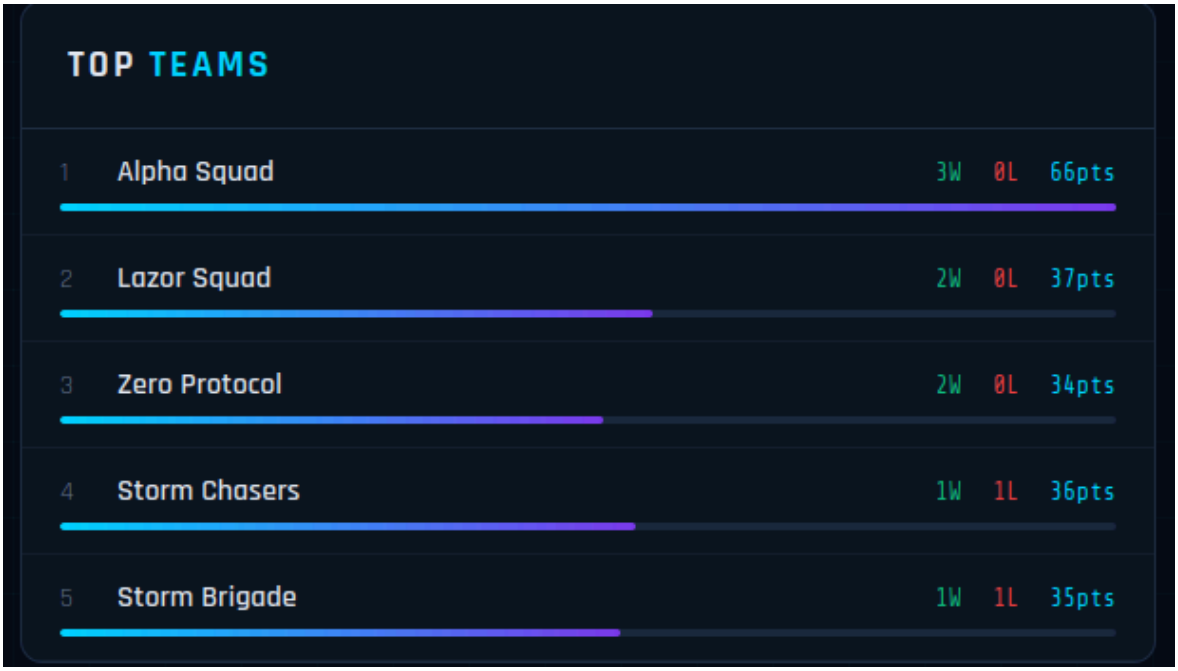
```sql
-- ============================================================
-- SELECT QUERIES
-- ============================================================

-- Q1: All players
SELECT * FROM Player;

-- Q2: INNER JOIN - matches with team names and tournament
SELECT m.match_id,
       t1.team_name AS team_1,
       t2.team_name AS team_2,
       tn.tournament_name,
       m.match_date,
       tw.team_name AS winner
FROM Match m
INNER JOIN Team t1 ON m.team1_id  = t1.team_id
INNER JOIN Team t2 ON m.team2_id  = t2.team_id
INNER JOIN Tournament tn ON m.tournament_id = tn.tournament_id
INNER JOIN Team tw ON m.winner_team_id = tw.team_id;
```

	match_id	team_1	team_2	tournament_name	match_date	winner
1	1	Alpha Squad	Blade Runners	PUBG Winter Cup 2025	2025-01-12 18:00	Alpha Squad
2	2	Storm Chasers	Night Wolves	PUBG Winter Cup 2025	2025-01-13 19:00	Storm Chasers
3	3	Zero Protocol	Ghost Unit	Valorant Spring Open	2025-03-07 20:00	Zero Protocol
4	4	Turbo Force	Lazor Squad	Valorant Spring Open	2025-03-08 18:00	Lazor Squad
5	5	Storm Brigade	Void Breakers	FIFA Champions League	2025-04-03 17:00	Storm Brigade
6	6	Alpha Squad	Storm Chasers	COD Summer Showdown	2025-06-17 21:00	Alpha Squad
7	7	Blade Runners	Zero Protocol	PUBG Pro Series	2025-08-03 19:00	Zero Protocol
8	8	Ghost Unit	Turbo Force	Valorant Elite Series	2025-09-12 20:00	Ghost Unit
9	9	Lazor Squad	Storm Brigade	PUBG Grand Prix	2026-01-07 18:00	Lazor Squad
10	10	Void Breakers	Alpha Squad	Valorant World Qualifier	2026-03-03 19:00	Alpha Squad

[Screenshot 5: Web front-end — Player registration form]



[Screenshot 6: Web front-end — Leaderboard display page]



[Screenshot 7: Aggregate function output — Average scores per team]

```sql
-- Q5: Aggregate - avg score per team
SELECT t.team_name,
       COUNT(s.score_id)     AS matches_played,
       SUM(s.points_scored)  AS total_points,
       ROUND(AVG(s.points_scored),2) AS avg_score,
       SUM(s.is_winner)      AS wins
FROM Score s
INNER JOIN Team t ON s.team_id = t.team_id
GROUP BY t.team_name
ORDER BY wins DESC, total_points DESC;
```

	team_name	matches_played	total_points	avg_score	wins
1	Alpha Squad	3	66	22	3
2	Lazor Squad	2	37	18.5	2
3	Zero Protocol	2	34	17	2
4	Storm Chasers	2	36	18	1
5	Storm Brigade	2	35	17.5	1
6	Ghost Unit	2	29	14.5	1
7	Turbo Force	2	23	11.5	0
8	Blade Runners	2	20	10	0
9	Void Breakers	2	20	10	0
10	Night Wolves	1	9	9	0

[Screenshot 8: Subquery result — Unbeaten teams] : ***optional***

```sql
SELECT team_name FROM Team
WHERE team_id IN (
    SELECT team_id FROM Score GROUP BY team_id HAVING MIN(is_winner) = 1
);
```

	team_name
1	Alpha Squad
2	Zero Protocol
3	Lazor Squad

8. Challenges

The following challenges were encountered during the development of this project:

Many-to-Many Relationships

Modeling the Player-Team relationship was initially confusing because a player can belong to multiple teams across different tournaments. Resolving this required creating the Team_Player bridge table with a composite primary key, which was a new concept to apply in practice.

Self-Referencing Foreign Key in Match Table

The Match table references the Team table twice (team1_id and team2_id). Writing JOIN queries on a table that references the same table twice required using table aliases, which took time to understand and implement correctly.

Normalization Decisions

Deciding which attributes to remove during 3NF normalization required careful analysis of functional dependencies. Some columns seemed harmless to leave in place but were actually causing transitive dependencies.

Front-End Database Integration

Connecting the web front-end to MySQL required configuring a local server environment (XAMPP) and learning how to sanitize user inputs to prevent SQL injection vulnerabilities.

9. Lessons Learned

This project provided valuable hands-on experience with the following concepts:

1. ERD design is the most critical step — a well-designed ERD makes every subsequent SQL query logical and straightforward. Errors in the ERD are expensive to fix later.
2. Normalization eliminates real problems. Before normalization, updating a team's name would have required changing it in dozens of rows. After normalization, it requires changing exactly one row in the Team table.
3. VIEWs are powerful tools for simplifying complex queries. The Leaderboard VIEW means any application can simply call SELECT * FROM Leaderboard without knowing the underlying JOIN logic.
4. JOINs are the backbone of relational databases. The ability to combine Player, Team, Match, and Score data in a single query is what makes a relational database far superior to isolated spreadsheets.
5. Subqueries enable answering questions that would be impossible in a single flat query — such as finding players on the team that won the most expensive tournament.
6. Data integrity through foreign keys prevents orphan records. The database itself enforces rules, not just the application.

10. Conclusion

The Gaming Tournament Tracker successfully demonstrates a complete relational database management system built from the ground up. Starting from an Entity Relationship Diagram, through relational schema design and 3NF normalization, to full SQL implementation with joins, aggregate functions, subqueries, and views, the project covers the entire spectrum of DBMS concepts covered in this course.

The system is practical, scalable, and solves a real organizational problem faced by gaming communities. Any esports club or tournament organizer could use this system to replace manual tracking with a reliable, consistent, and query-able database.

The front-end interface makes the system accessible to non-technical users, while the underlying SQL logic ensures data integrity and efficient retrieval. This project has reinforced the importance of careful upfront design, the power of normalization, and the elegance of SQL as a query language.

The experience gained from this project — particularly in ERD design, normalization reasoning, and multi-table SQL queries — forms a strong foundation for future work in database administration, back-end development, and data engineering.

11. References and Resources

Official Documentation

- Microsoft T-SQL Reference: https://learn.microsoft.com/en-us/sql/t-sql/language-reference
- SQL Server Data Types: https://learn.microsoft.com/en-us/sql/t-sql/data-types/data-types-transact-sql
- SQL Server on Docker: https://learn.microsoft.com/en-us/sql/linux/quickstart-install-connect-docker
- VS Code mssql Extension: https://marketplace.visualstudio.com/items?itemName=ms-mssql.mssql
- Beekeeper Studio Documentation: https://docs.beekeeperstudio.io

DBMS Concepts

- Entity Relationship Modelling — Ramez Elmasri & Shamkant Navathe, "Fundamentals of Database Systems", 7th Edition, Pearson
- Normalization (1NF/2NF/3NF) — C.J. Date, "An Introduction to Database Systems", 8th Edition, Addison-Wesley
- SQL Joins explained visually: https://www.w3schools.com/sql/sql_join.asp
- SQL Aggregate Functions: https://www.w3schools.com/sql/sql_aggregate_functions.asp
- SQL Subqueries: https://www.w3schools.com/sql/sql_subqueries.asp
- SQL Views: https://www.w3schools.com/sql/sql_view.asp

Tools Used

- draw.io (diagrams.net) — ERD design: https://app.diagrams.net
- MySQL Workbench — ERD and schema visualization: https://www.mysql.com/products/workbench
- Docker — SQL Server container: https://hub.docker.com/_/microsoft-mssql-server
- VS Code — Code editor: https://code.visualstudio.com
- Beekeeper Studio — Database GUI: https://www.beekeeperstudio.io
- Google Fonts (Rajdhani, Exo 2) — Web front-end typography: https://fonts.google.com

AI Assistance

This project structure, framework, and front-end interface developed with the assistance of Claude (claude.ai) by Anthropic.